Cape Peninsula University of Technology

Faculty of Engineering and the Built Environment

Department of Electrical Electronic and Computer Engineering

Time Matters Appointment Scheduler Technical Report

Software Design 1 SDN150S | Project 12

Prepared for Management | Group 12 | Bellville campus

Submission date 8 November 2026

Reneloe Phakwago 260633283

Zilungile Cwathi 250513447

Liyabona Ndevu 260173436

Phakamani Dlamini 261059742

Noluthando Basi 260874574

Lisakhanya Songqwaba 261136437

Okuhle Ntweni 260016039

Sekete Manthata 260555770

Repository: https://github.com/ZuluBarbie/AppointmentScheduler

Abstract

Time Matters is a Java appointment scheduler that helps users organise their time through a graphical interface. It supports creating, editing and deleting appointments, daily, weekly and monthly views, reminders, recurrence, conflict detection and calendar export. The application uses Java Swing, a NetBeans JFrame Form and local file storage. This report explains the problem, requirements, design approach, implementation, verified results and limitations. The Windows builds succeeded and all 64 automated checks passed.

1 Introduction

Time management is important for students and professionals. Paper diaries and informal notes can be lost, become difficult to organise or leave appointments overlooked. A digital scheduler brings appointment details and reminders into one place. Our Group 12 project addresses the Appointment Scheduler requirements in the Software Design 1 brief (Cape Peninsula University of Technology, 2026). The intended benefit is a simple personal scheduling tool that preserves appointments between sessions.

1.1 Problem statement and aim

Users need a convenient way to organise appointments without relying on scattered notes. Our aim is to implement a Java desktop scheduler with a readable interface, validated appointment details, persistent storage and timely reminders. The required operations are creation, editing, deletion and viewing by day, week or month.

1.2 Sub questions

How can one stored appointment series support different views? How can invalid input and conflicting times be rejected safely? How can reminders alert once per occurrence during a session? How can the displayed schedule be transferred to another calendar?

2 Requirements and scope

Table 1 summarises the verified implementation. Login is outside the project scope; the welcome screen opens the scheduler directly.

Requirement	Implemented behaviour
Create appointments	Validated title, date, start and end times, and optional location.
Edit and delete	Select a row to edit its details or confirm deletion. A recurring row represents its whole series.
Schedule views	Day, Monday-to-Sunday Week and calendar Month filter and sort dated occurrences.
Persistent storage	Local appointments.tsv records survive application restarts.
Reminders	A Swing timer displays due alerts while the scheduler remains open.
Recurrence and conflicts	Daily and weekly series have an inclusive end date. Overlapping occurrences are rejected.
Calendar export	Displayed occurrences can be exported to an ICS file for calendar import.

Table 1. Implemented appointment workflow and extensions.

3 Software design approach

The application uses an event-driven interface. Swing components such as JFrame, JLabel, JTextField, JButton and JTable display information and accept input. Button listeners perform the corresponding operations. Main opens NewJFrame1, the Time Matters welcome JFrame. Open scheduler creates SchedulerFrame. SchedulerFrame has a paired NetBeans visual form; the welcome layout is constructed in Java using GridBagLayout. AppTheme applies the shared cyan branding and light scheduler workspace.

The design separates interface behaviour from appointment rules. Appointment holds validated details. AppointmentBook manages storage and views. Occurrence links a displayed date to its saved appointment. Reminders tracks alerts, while CalendarExport writes calendar files. Constructors initialise these objects, private modifiers protect state and public methods expose operations. Loops, conditions and switch expressions implement validation and date filtering.

3.1 Data storage

AppointmentBook maintains a list of structured appointment records and loads appointments.tsv from the working directory. The NetBeans and standalone launchers use their own working directories. Strings are encoded for safe text storage. A change is written to a temporary file before replacement of the saved file. The book refuses to overwrite data changed by another open window (Oracle, n.d.). A recurring appointment is stored once and expanded into dated occurrences for the selected view.

3.2 Input validation and error handling

Validation rejects blank or overlong titles, invalid dates and times, an end time not after its start, and invalid reminder or recurrence values. Repeat end dates must be on or within 366 days of the start date. Conflict checking compares occurrence dates and time intervals; adjacent appointments are permitted. Errors are reported visibly, and unsuccessful changes do not replace the saved records.

4 Engineering method skills and tools

An incremental method was used: basic window, appointment entry, editing and storage, reminders and recurrence, then export. NetBeans 31 provided source editing and JFrame Form design. JDK compilation targets Java 21 using release compilation. GitHub records meaningful development updates. Skills applied included object-oriented decomposition, Swing event handling, date and time calculations, file handling, debugging and regression testing. Calendar export follows the iCalendar specification for escaping, UTC timestamps and line folding (Desruisseaux, 2009).

5 Pseudocode

1. Start the application and show the Time Matters welcome window.

2. When Open scheduler is selected, open the scheduler and close the welcome window.

3. Load saved appointments. If loading fails, preserve the file, display the error and disable saving.

4. Display appointments for the selected day, week or month in date and start-time order.

5. When the view or navigation date changes, calculate its range and refresh the displayed occurrences.

6. To add, read the title, date, times, location, recurrence and reminder settings.

7. Reject invalid or missing values and repeat ranges outside the allowed limits.

8. Compare candidate occurrences with other appointments. Reject overlapping times on a shared date.

9. If valid, save the changed list safely and refresh the view. Report write errors without losing the existing records.

10. When a row is selected, populate the entry fields and enable editing and deletion.

11. To edit, validate the replacement and check conflicts while excluding the original series. Save and refresh.

12. To delete, request confirmation, remove the selected appointment or whole series, save and refresh.

13. To clear the selection, reset the fields without deleting saved appointments.

14. While the scheduler remains open, check upcoming occurrences every second. Show each due reminder once per session.

15. To export, copy the displayed occurrences and request a destination. Confirm replacement if needed.

16. Write one calendar event per occurrence. Display success or an error message.

17. When the scheduler closes, stop its reminder timer.

6 UML class diagram

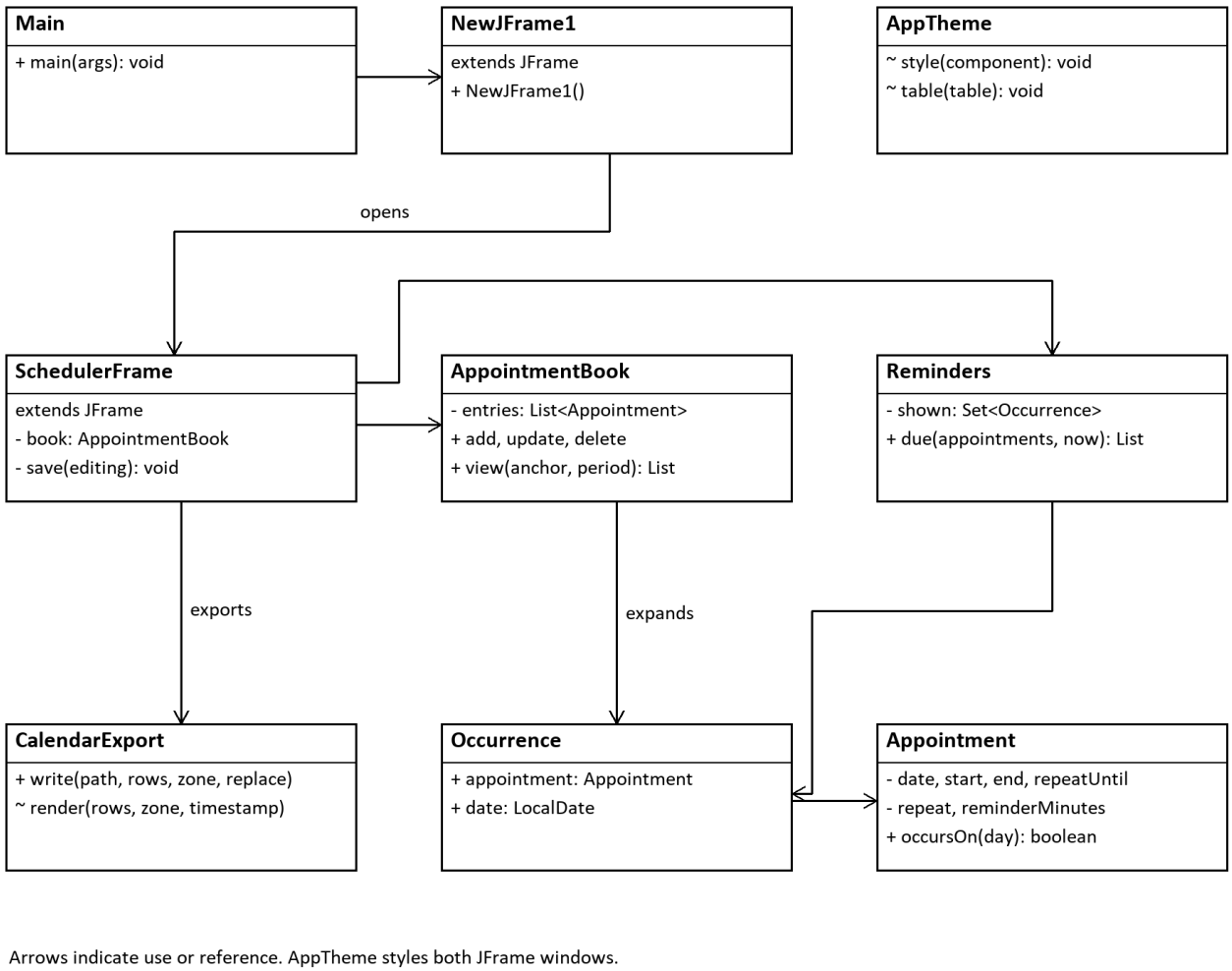


Figure 1. The welcome JFrame opens SchedulerFrame, which uses separate storage, reminder and export classes. Each Occurrence references its Appointment.

7 Results and analysis

The NetBeans Ant build and standalone build succeeded on Windows. All 64 automated checks passed on 7 November. They cover input boundaries, persistence, failed writes, date filtering, recurrence, conflict rejection, reminder timing and calendar export. Swing integration checks confirmed welcome navigation, saved appointment loading, all three views and selection for editing. Verification preserved the normal data files.

Earlier export verification exercised the save chooser and inspected the resulting ICS file. Import into a third-party calendar was not verified. The welcome and scheduler windows were rendered and inspected after styling changes. These results support the implemented workflow within the tested local Windows environment.

8 Limitations

Reminders require the application to remain open and can appear again after reopening within the reminder window. Editing or deleting a recurring row affects its entire series. Local storage is intended for one user; the application does not provide login or shared online access. Export creates a calendar snapshot, and repeated imports may create duplicates. A damaged file must be corrected or restored before saving resumes.

9 Conclusion

Time Matters implements the required appointment operations, schedule views and reminders, with validation and persistent storage. Recurrence, conflict detection and export extend the core requirements. The design separates interface and business logic and uses the specified Java concepts. The complete Java listing and editable source project accompany this report. Together, these materials document the application and its verified behaviour for management.

10 Java program

The complete program is supplied in TimeMatters_Complete_Java_Program.pdf and TimeMatters_NetBeans_Source.zip. The entry point is demo.Main. The following excerpt from Appointment.java shows how the program determines whether a stored appointment occurs on a selected date.

```
public boolean occursOn(LocalDate day) {
   if (day.isBefore(date) || day.isAfter(repeatUntil)) return false;
   return repeat != Repeat.WEEKLY
      || day.getDayOfWeek() == date.getDayOfWeek();
}
```

11 References

Cape Peninsula University of Technology (2026) Software Design 1 SDN150S project brief. Course handout.

Desruisseaux, B. (ed.) (2009) Internet Calendaring and Scheduling Core Object Specification (iCalendar). RFC 5545. Available at: https://www.rfc-editor.org/rfc/rfc5545 (Accessed: 6 November 2026).

Oracle (n.d.) Files. Java SE 21. Available at: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/nio/file/Files.html (Accessed: 4 November 2026).